

Algorithmes et Pensée Computationnelle

Fonctions, méthodes, mémoire et exceptions

Le but de cette séance est d'approfondir vos connaissances en programmation. Au terme de cette séance, vous serez capable de :

- définir une fonction en Python ou une méthode en Java et l'utiliser dans un programme,
- utiliser des bibliothèques contenant des fonctions prédéfinies en Python ou des méthodes prédéfinies en Java,
- connaître la portée d'une variable,
- gérer des exceptions.

 Pour les exercices où vous devez déterminer ce qu'affiche un programme, résolvez-les d'abord à la main avant d'utiliser IntelliJ, afin de simuler les conditions d'examen.

1 Opérations arithmétiques

Question 1: (🕒 3 minutes) [Java et Python] Calculs (multiplication)

Créez deux variables nommées `facteur1` et `facteur2` en Java (`facteur_1` et `facteur_2` en Python), avec pour valeurs respectives 11 et 3. Multipliez la première variable par la deuxième et stockez le résultat dans une nouvelle variable `produit`. Vous pouvez afficher les différentes variables pour voir leurs valeurs. Vous pouvez répéter l'exercice avec l'addition et la soustraction.

Conseil

L'opérateur de multiplication est le `*`, celui de l'addition est le `+` et celui de soustraction est le `-`.

Question 2: (🕒 10 minutes) [Java et Python] Calculs (division)

Créez deux variables nommées `nbBonbons` et `nbPersonnes` en Java (`nb_bonbons` et `nb_personnes` en Python), avec pour valeurs respectives 11 et 3. Divisez la première variable par la deuxième et stockez le résultat dans une nouvelle variable nommée `bonbonsPersonnes` en Java (`bonbons_personnes` en Python). Pour finir, calculez le nombre de bonbons restants via l'opérateur `%` (modulo) et stockez le résultat dans une nouvelle variable `reste`. Vous pouvez afficher les différentes variables pour voir leurs valeurs. Chaque personne doit recevoir un nombre entier de bonbons.

Conseil

- Attention : en Python il existe deux opérateurs de division, `/` effectue une division classique, tandis que `//` effectue une division avec arrondi inférieur.
- En Java, lorsque les deux opérandes sont de type `int`, l'opérateur `/` effectue une division entière. Si au moins l'un des deux opérandes est de type `float` ou `double`, il effectue une division en virgule flottante.
- L'opérateur `%` (modulo) permet de calculer le reste d'une division. Par exemple, `10%2=0`.

Question 3: (🕒 5 minutes) [Java et Python] Calculs (incrément / décrémentation)

Gardez vos variables de l'exercice précédent, augmentez de 1 la valeur de `nbBonbons` en Java (`nb_bonbons` en Python) et diminuez de 1 celle de `nbPersonnes` en Java (`nb_personnes` en Python).

Conseil

Vous pouvez utiliser les opérateurs `+=` et `-=` en Python, et les opérateurs `++` (incrément) et `--` (décrément) en Java.

2 Variables, fonctions et méthodes

Question 4: (🕒 5 minutes) [Java et Python] Fonctions et méthodes basiques

Définissez une fonction en Python et une méthode en Java nommées `ping()` qui, lorsqu'elles sont appelées, affichent `pong`.

💡 Conseil

Référez-vous aux diapositives dans [ce dossier Moodle](#) pour la création des fonctions en Python et des méthodes en Java.

Question 5: (🕒 5 minutes) [Java et Python] Fonction et méthode de multiplication

Définissez une fonction en Python et une méthode en Java nommées `multiplicateur()` qui prennent deux arguments `multiple1` et `multiple2`, les multiplient et retournent le résultat. Stockez le résultat de `multiplicateur(2, 3)` dans une variable `resultat` et affichez-la.

💡 Conseil

- Référez-vous aux diapositives du cours pour la création et l'appel des fonctions en Python et des méthodes en Java.
- Pour retourner une valeur au lieu de l'afficher, utilisez le mot-clé `return` (pour Python et Java).

Question 6: (🕒 5 minutes) [Java et Python] Fonctions et méthodes pour l'aire et le périmètre

Définissez deux fonctions en Python et deux méthodes en Java nommées `aire()` et `perimetre()`. Elles prennent un argument (`rayon`) et renvoient respectivement l'aire et le périmètre d'un cercle. Stockez les résultats dans des variables `aire_resultat` et `perimetre_resultat` et affichez le contenu de ces variables.

💡 Conseil

- Référez-vous aux diapositives du cours pour la création et l'appel des fonctions en Python et des méthodes en Java.
- Pour retourner une valeur au lieu de l'imprimer, utilisez le mot-clé `return` (pour Python et Java).
- Pour calculer a^b en Python, utilisez `a**b`, et `Math.pow(a, b)` en Java.
- Pour rappel, le périmètre d'un cercle s'obtient en utilisant la formule $P = 2 * \pi * r$ et l'aire s'obtient en utilisant la formule $A = r^2 * \pi$.

Question 7: (🕒 10 minutes) [Java] Portée des variables

Qu'affiche le programme suivant ?

```
1 public class Question7 {
2     static int a = 0;
3     public static void f() {
4         a += 2;
5         System.out.println("a = " + a);
6     }
7
8     public static void g() {
9         int b = 3;
10        System.out.println("a = " + a);
11        System.out.println("b = " + b);
12    }
```

```

12     }
13
14     public static void main(String[] args) {
15         f();
16         g();
17         //System.out.println("b =" + b);
18     }
19 }

```

Que se passerait-il si on décommente la ligne 17 (c'est-à-dire si l'on enlève les //)?

Conseil

Tenir compte de l'endroit où chaque variable est définie ainsi que des opérations réalisées sur chaque variable.

Question 8: (🕒 5 minutes) [Python] Portée des variables

Qu'affiche le programme suivant ?

```

1 x = 2
2
3 def fonction():
4     x = 3
5
6     def fonction2():
7         global x
8         print("x=" + str(x))
9
10    fonction2()
11    print("x=" + str(x))
12
13 print(fonction())

```

Conseil

- Le mot-clé `global` permet d'accéder aux variables globales (définies à l'extérieur de la fonction).
- Soyez attentif au type des éléments retournés par chacune des fonctions.
- En Python, une fonction qui atteint la fin de son exécution sans rencontrer d'instruction `return`, ou qui exécute une instruction `return` sans expression, renvoie implicitement `None`.

3 Gestion des exceptions

Question 9: (🕒 5 minutes) [Python] Types d'erreurs Qu'affiche le programme suivant ?

```

1 valeur1 = "Algorithms"
2 valeur2 = 4
3
4 try:
5     size = len(valeur1)
6     result = size/valeur2
7     print(f"Le résultat de la division est: {result}")
8

```

```

9 except Exception as error:
10     print("On ne peut pas effectuer l'opération")
11
12 try:
13     result = value1/2
14     print(f"Le résultat de la division est: {result}")
15
16 except TypeError as error:
17     print("On ne peut pas diviser une chaîne de caractères")

```

Conseil

Utilisée sur une chaîne de caractères, la fonction `len()` renvoie le nombre de caractères de la chaîne.

Question 10: (🕒 5 minutes) [Python] **Types d'erreurs** Qu'affiche le programme suivant ?

```

1 value1 = 4
2 value2 = ""
3
4 try:
5     count = len(value2)
6     result= value1/count
7 except ZeroDivisionError as error:
8     print("Nous ne pouvons pas diviser un nombre par 0")

```

Conseil

La chaîne de caractères vide est représentée par ""

Question 11: (🕒 5 minutes) [Python] **Types d'erreurs** Qu'affiche le programme suivant ?

```

1 value1 = "Algorithms"
2 value2 = 4
3
4
5 try:
6     decimal = float(value2)
7 except ValueError as error:
8     print("Nous ne pouvons pas convertir un entier en décimal")
9
10 finally:
11     try:
12         value2 = int(value1)
13     except ValueError as error:
14         print("Nous ne pouvons pas convertir une chaîne de caractères
    en nombre")

```

Conseil

- `TypeError` survient lorsqu'une opération est effectuée sur des types incompatibles (exemple : `"abc" + 5`), tandis que `ValueError` survient lorsque le type est approprié, mais que la valeur est invalide (exemple : `int("abc")`).
- La fonction `float()` permet de convertir une variable en nombre à virgule flottante.
- La fonction `int()` permet de convertir une variable en nombre entier.

Question 12: (🕒 5 minutes) [Java] **Gestion d'erreurs** Le programme suivant est incorrect. Que devez-vous modifier pour qu'il fonctionne correctement ?

Conseil

Référez-vous à la diapositive 24 du cours de cette semaine.

```
1 public class Question12 {
2
3     public static void division(int a, int b) {
4         if (b == 0) {
5             throw new ArithmeticException();
6         } else {
7             float result = a / b;
8             System.out.println("Le résultat de la division de " + a +
9             "/" + b + " = " + result);
10        }
11    }
12
13    public static void main(String args[]) {
14        int value1 = 2;
15        int value2 = 4;
16        try {
17            division(value1, value2);
18            value2 = 0;
19            division(value1, value2);
20        } catch (IndexOutOfBoundsException err) {
21            System.out.println("Nous ne pouvons pas effectuer une
22            division par 0");
23        }
24    }
25 }
```

Question 13: (🕒 5 minutes) [Java] **Gestion d'erreurs**

1. Qu'affiche le programme suivant, en supposant que l'utilisateur saisit les entrées suivantes ?

- Entrez le premier entier : 12
- Entrez le deuxième entier : 2
- Entrez l'opération (+, -, *, /) : *

Conseil

Consultez [ce document](#) sur le `switch` en Java. Référez-vous à la diapositive 23 du cours.

2. Que devez-vous modifier pour qu'il fonctionne ?

```

1  import java.util.Scanner;
2
3  public class Calculator {
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6
7          System.out.print("Entrez le premier entier: ");
8          double num1 = sc.nextInt();
9
10         System.out.print("Entrez le deuxième entier: ");
11         double num2 = sc.nextInt();
12
13         System.out.print("Entrez l'opération (+, -, *, /): ");
14         char op = sc.next().charAt(0);
15
16         sc.close();
17
18         switch (op) {
19             case '+':
20                 System.out.println(num1 + num2);
21             case '-':
22                 System.out.println(num1 - num2);
23             case '*':
24                 System.out.println(num1 * num2);
25             case '/':
26                 System.out.println(num1 / num2);
27             default:
28                 throw new IllegalArgumentException("L'opération n'est
pas valide");
29         }
30     }
31 }

```

Question 14: (🕒 5 minutes) Qu'affiche le programme suivant, en supposant que l'utilisateur exécute le programme de la manière suivante dans le terminal ?

```
python3 question14.py 9
```

💡 Conseil

Référez-vous à la diapositive 10 du cours.

```

1  import sys
2  if __name__ == '__main__':
3      balance = 20
4      try:
5          balance = balance - sys.argv[1]
6          solde_post = balance > 0
7          match (solde_post):
8              case True:
9                  print('Le solde de votre compte est positif.')
10             case False:
11                 print('Le solde de votre compte est négatif ou zéro.')
12                 raise ValueError('Le compte ne peut pas être débité')
13     except Exception as e:
14         print(f"Une erreur est survenue: {e}")
15     finally:

```

```
print(f"Valeur finale de balance: {balance}")
```

Question 15: (🕒 5 minutes) Laquelle des affirmations suivantes est **fausse** concernant la différence entre une Error (erreur) et une Exception (exception) en Java ?

💡 Conseil

Référez-vous à la diapositive 28 du cours.

- (A) Les erreurs sont généralement causées par l'environnement dans lequel l'application s'exécute, tandis que les exceptions sont généralement causées par l'application elle-même.
- (B) Les erreurs sont non vérifiées (*unchecked*) et n'ont pas besoin d'être déclarées dans la clause `throws` d'une méthode, tandis que les exceptions peuvent être soit vérifiées (*checked*), soit non vérifiées (*unchecked*).
- (C) Les erreurs devraient généralement être interceptées et gérées par l'application afin d'assurer une récupération fluide, tandis que les exceptions peuvent être récupérables ou non.
- (D) Les erreurs indiquent généralement des problèmes graves qu'une application raisonnable ne devrait pas essayer d'intercepter.
- (E) `RuntimeException` sont des exceptions non vérifiées (*unchecked*).